

[Appendix] SLeDGe: Semi-Supervised Learning on Data Streams via Graph Structure Learning

A Datasets

Table 1 summarizes the statistics of all datasets in our experiments.

Web application: We use 3 web application datasets: MNDS [7], Shopper [8], and WebKB [5], extracting 500 numerical features from the text of MNDS and WebKB using TfidfVectorizer.

Image: We use 4 image datasets: MNIST [2], CIFAR-10 [6], KMNIST [1], and F.MNIST (FashionMNIST) [10], configuring the dataloader with a batch size of 1 to emulate a data-streaming setting.

Tabular: We use 5 tabular datasets from UCI Repository¹: Statlog (Shuttle), GSD (Gas Sensor Array Drift), SLDD (Sensorless Drive Diagnosis), HAR (Human Activity Recognition Using Smartphones), and Occup. (Occupancy Detection).

B Time and Space Complexity Analysis

Memory Update: Updating the labeled memory requires $O(1)$ time. For the unlabeled memory, when it reaches full capacity, selecting the most similar prototype requires comparing the new sample to all U stored prototypes via cosine similarity, resulting in a cost of $O(U^2h)$. Since the memory module stores only the labeled and unlabeled samples, its total space footprint is $O(Ld + Ud) = O(Md)$, where $M = L + U$.

Dynamic Graph Inference: The embedding function f_E incurs a cost of $O(\alpha_1 Mh^2)$, while the scoring function f_S requires $O((M + 1)^2h)$ due to pairwise similarity computation. The pruning function f_P takes $O(M^2)$ time. Thus, the total time complexity of the graph inference step is $O(\alpha_1 Mh^2 + M^2h)$. All latent embeddings and adjacency matrices are computed on the fly at each time step and discarded afterward, contributing no additional space overhead. The learnable parameters of f_E require $O(\alpha_1 h^2)$ space.

Graph-enhanced Classifier: Given a sparse adjacency matrix with at most $k(M + 1)$ edges, message passing and the α_2 -layer MLP incur a total computational cost of $O(\alpha_2 Mh^2)$. The space required to store the *Backbone* model parameters is $O(\alpha_2 h^2)$.

Overall Complexity: Combining all components, the dominant computational cost per time step is

$$O(M^2h + \alpha_1 Mh^2 + \alpha_2 Mh^2).$$

The total space complexity is determined by the memory module and the learnable parameters of both f_E and the *Backbone* model, yielding

$$O(Md + \alpha_1 h^2 + \alpha_2 h^2).$$

C Implementation Details

C.1 Experimental Environment

We conduct all experiments with RTX A6000 GPUs (48GB VRAM), 512GB of RAM, and Intel Xeon Silver 4210R Processors.

C.2 General Setup

We assume no initial labeled training set. The total memory size is fixed at 1,000 samples. For labeled data proportions of 0.1% and

¹<https://archive.ics.uci.edu/>

Table 1: Dataset Statistics

Dataset	$ f $	$ c $	$ D $
MNDS	500	17	10,917
Shopper	15	2	12,330
WebKB	500	7	8,282
MNIST	784	10	70,000
CIFAR-10	3,072	10	60,000
KMNIST	784	10	70,000
F.MNIST	784	10	70,000
Shuttle	7	7	58,000
GSD	128	6	13,910
SLDD	48	11	58,509
HAR	561	6	10,299
Occup.	5	2	20,560

1.0%, we allocate 10 and 100 samples for labeled data, respectively. The remainder of the memory (990 and 900 samples, respectively) is reserved for unlabeled data.

For all deep learning models, we perform a hyperparameter search for the learning rate (lr) and weight decay (wd). The search space for the learning rate is $lr \in \{1e^{-5}, 5e^{-5}, 1e^{-4}, 5e^{-4}\}$ and for weight decay is $wd \in \{0.0, 5e^{-4}\}$.

C.3 Baselines

TLP [9]: is a graph-based semi-supervised learning method. We adopt the labeled and unlabeled sample counts as described above and followed the algorithm specified in the original paper.

ReSSL [4]: is a cluster-based semi-supervised learning method. We use the authors' official implementation with its default hyperparameters, unless otherwise specified. The memory size is set as described in our general setup.

HDSSL [3]: is also a cluster-based semi-supervised learning method. Since we do not have an initial labeled training set, the Denoising Autoencoder (DAE) component could not be trained; therefore, we use only its clustering component. We set "chunkSize" = 1 and vary the number of neighbors $k \in \{1, 3, 5, 10\}$. All other hyperparameters follow the original implementation.

C.4 Model architectures and ablations

Backbone (Base): Our backbone model is an α_2 -layer MLP with layer normalization ($\alpha_2 \in \{1, 2, 3\}$), chosen for its flexibility. This model uses the memory modules by default.

Without memory (-M): For this ablation, we use only the α_2 -layer MLP without any memory modules. Labeled samples update the model via single-instance training, and unlabeled samples are used only for evaluation.

+Graph: This variant incorporates a k -nearest neighbors (kNN) graph for information propagation. The graph is constructed using cosine similarity on the static input features. We search for the balancing parameter $\lambda \in \{0.001, 0.01, 0.1, 1.0\}$ and the number of neighbors $k \in \{1, 5, 10\}$.

Table 2: Average running time.

Label	0.1%				1.0%			
Dataset	TLP	ReSSL	HDSSL	SLeDGe	TLP	ReSSL	HDSSL	SLeDGe
MNDS	111.0	109.4	18.8	234.6	114.2	113.2	83.9	201.1
	111.9	35.9	14.1	210.4	107.7	36.5	20.2	220.3
	79.7	90.1	22.2	149.1	76.9	88.9	98.4	141.0
MNIST	698.4	925.1	404.0	1137.0	717.6	1050.1	6637.3	1037.4
CIFAR-10	617.4	3634.0	639.0	709.4	630.9	3477.9	5910.3	829.5
KMNIST	736.3	876.6	367.7	1015.6	745.0	1082.9	5406.5	1208.2
F.MNIST	722.0	872.1	678.9	1039.9	722.9	1035.5	6370.1	985.7
Shuttle	542.4	158.4	94.5	939.1	554.6	160.8	177.7	638.5
GSD	129.1	69.3	29.3	191.5	131.3	68.6	74.7	184.3
SLDD	571.0	214.2	97.8	889.1	580.7	220.9	455.1	921.9
HAR	95.0	106.5	22.9	131.2	95.9	116.7	147.8	133.6
Occup.	185.3	61.9	31.7	209.9	187.6	61.5	36.6	311.0

+GSL (SLeDGe): For graph structure learning (GSL), we use an α_1 -layer MLP with layer normalization as the embedding function, with $\alpha_1 \in \{1, 2, 3\}$. The balancing parameter λ is selected from $\{0.001, 0.01, 0.1, 1.0\}$ and the number of $k \in \{1, 5, 10\}$.

D Additional Experiment Results

D.1 Running Time

We compare the running time of each method, reporting the time for its best-performing hyperparameter configuration.

It is important to view this comparison qualitatively, as the methods were run in different environmental settings (e.g., CPU vs. GPU) and implemented in different languages (Python vs. Matlab vs. Java). Both of these factors significantly impact execution speed.

Theoretical Time Complexity: To analyze theoretical scalability, we compare the dominant time complexities of the methods:

- **TLP:** $O(M^3 + Md)$
- **ReSSL:** $O(M^2d)$
- **HDSSL:** $O(M^2d)$
- **SLeDGe:** $O(M^2h + \alpha_1 Mh^2 + \alpha_2 Mh^2)$

The key difference lies in the dependency on the input feature dimension d . The complexities of ReSSL and HDSSL scale linearly with d . In contrast, the complexity of SLeDGe is dependent on the hidden dimension h , not d .

Empirical Runtime: As shown in Table 2, SLeDGe incurs a higher computational cost than the baseline methods. This is an expected overhead, as SLeDGe is the only deep learning model evaluated, whereas the baselines are not.

However, SLeDGe’s runtime is not the slowest in all cases. Its efficiency is particularly competitive when the input feature dimension is much larger than the hidden dimension ($d \gg h$). This tendency is observed on the CIFAR-10 dataset, which has a relatively large input dimension: SLeDGe runs faster than both ReSSL and HDSSL when the label ratio is 1.0%.

D.2 Memory Size Analysis

We analyze the model’s performance, varying the labeled memory size L and the unlabeled memory size U . To isolate the impact of each component, we vary only one size (L or U) at a time, keeping all other hyperparameters constant. Our baseline configuration uses $L = 100$ and $U = 900$ (reflecting the 1.0% label ratio setting). We test variations by independently halving (e.g., $L = 50$ or $U = 450$)

Table 3: SLeDGe resultson labeled (L) and unlabeled (U) memory sizes. Datasets shaded in gray perform optimally at $\tau = 1$, while all others perform optimally at $\tau = 100$.

Memory	Labeled memory, M_L			Unlabeled memory, M_U		
Dataset	50	100	200	450	900	1800
MNDS	0.4455	0.4439	0.4437	0.4439	0.4439	0.4389
Shopper	0.8448	0.8448	0.8448	0.8448	0.8448	0.8448
WebKB	0.9060	0.9060	0.9060	0.9059	0.9060	0.9061
MNIST	0.8419	0.8527	0.8620	0.8363	0.8527	0.8628
CIFAR-10	0.2340	0.2508	0.2584	0.2467	0.2508	0.2520
KMNIST	0.7437	0.7645	0.7804	0.7426	0.7645	0.7825
F.MNIST	0.7071	0.7233	0.7345	0.7208	0.7233	0.7285
Shuttle	0.9713	0.9751	0.9797	0.9739	0.9751	0.9740
GSD	0.7923	0.7893	0.7873	0.7851	0.7893	0.7890
SLDD	0.9820	0.9819	0.9822	0.9819	0.9819	0.9820
HAR	0.7856	0.7916	0.7915	0.7844	0.7916	0.7803
Occup.	0.8862	0.8860	0.8851	0.8817	0.8860	0.8874

or doubling (e.g., $L = 200$ or $U = 1800$) these values. The results are summarized in Table 3.

While one might expect performance to correlate positively with memory size, our empirical findings show a more complex relationship that depends on the dataset’s optimal temporal weight τ . When the best performance is achieved at $\tau = 1$, it suggests that rapid adaptation to new inputs is critical. Conversely, when $\tau = 100$ is optimal, it implies that prioritizing memory and maintaining stable, long-term knowledge is more beneficial.

This finding directly connects to the impact of L . On datasets where $\tau = 1$ is optimal (prioritizing new data), performance is robust to reductions in the labeled memory size. On the Occup. dataset, decreasing L from 100 to 50 only negligibly changes performance (0.8860 $\rightarrow$ 0.8862).

In contrast, for datasets where $\tau = 100$ is optimal (prioritizing memory), performance is sensitive to L . On the CIFAR-10 dataset, increasing L from 100 to 200 improves performance significantly: 0.2508 $\rightarrow$ 0.2584.

In contrast to the conditional impact of L , the impact of the unlabeled memory size U is straightforward and aligns with our intuition. Increasing U consistently results in better performance. For instance, performance on the Occup. dataset improves (0.8860 $\rightarrow$ 0.8874), and on the CIFAR-10 dataset, it also increases (0.2508 $\rightarrow$ 0.2520).

References

- [1] Tarin Clanuwat, Mikel Bober-Irizar, Asanobu Kitamoto, Alex Lamb, Kazuaki Yamamoto, and David Ha. 2018. *Deep Learning for Classical Japanese Literature*. arXiv:cs.CV/1812.01718 [cs.CV]
- [2] Li Deng. 2012. The mnist database of handwritten digit images for machine learning research [best of the web]. *IEEE signal processing magazine* 29, 6 (2012), 141–142.
- [3] Salah Ud Din, Jay Kumar, Junming Shao, Cobbinah Bernard Mawuli, and Walidiodio David Ndiaye. 2021. Learning high-dimensional evolving data streams with limited labels. *IEEE Transactions on Cybernetics* 52, 11 (2021), 11373–11384.
- [4] Salah Ud Din, Junming Shao, Jay Kumar, Waqar Ali, Jiaming Liu, and Yu Ye. 2020. Online reliable semi-supervised learning on evolving data streams. *Information Sciences* 525 (2020), 153–171.
- [5] CMU Text Learning Group. 1998. The 4 Universities Data Set. <https://www.cs.cmu.edu/afs/cs.cmu.edu/project/theo-20/www/data/>. Last update: January 11 1998.
- [6] Alex Krizhevsky, Geoffrey Hinton, et al. 2009. Learning multiple layers of features from tiny images.(2009).
- [7] Alina Petukhova and Nuno Fachada. 2023. MN-DS: A multilabeled news dataset for news articles hierarchical classification. *Data* 8, 5 (2023), 74.

- [8] C. Sakar and Yomi Kastro. 2018. Online Shoppers Purchasing Intention Dataset. UCI Machine Learning Repository. DOI: <https://doi.org/10.24432/C5F88Q>.
- [9] Tal Wagner, Sudipto Guha, Shiva Kasiviswanathan, and Nina Mishra. 2018. Semi-supervised learning on data streams via temporal label propagation. In *International Conference on Machine Learning*. PMLR, 5095–5104.
- [10] Han Xiao, Kashif Rasul, and Roland Vollgraf. 2017. Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. arXiv:1708.07747 [cs.LG] <https://arxiv.org/abs/1708.07747>